

Tarea 1

Programación Funcional en Racket

Para la resolución de la tarea recuerde que

- Toda función debe estar acompañada de su firma, una breve descripción coloquial (en `T1.rkt`) y un conjunto significativo de tests (en `test.rkt`).
- Todo datatype definido por el usuario (via `deftype`) debe estar acompañado de una breve descripción coloquial y de la gramática BNF que lo genera.

Si la función o el datatype no cumple con estas reglas, será ignorado.

Ejercicio 1

60 Pt

Los polinomios de coeficientes enteros (ejemplo: $4x^5 + 3x^2 + 5$) pueden ser definidos de forma inductiva mediante las siguientes reglas:

$$\frac{}{\text{nullp} \in \text{Poly}} \quad \frac{\text{coef} \in \text{Integer} \quad \text{deg} \in \text{Integer} \quad \text{rem} \in \text{Poly}}{(\text{plus } \text{coef } \text{deg } \text{rem}) \in \text{Poly}}$$

Por ejemplo, el polinomio $4x^5 + 3x^2 + 5$ es representado por la construcción `(plus 4 5 (plus 3 2 (plus 5 0 (nullp))))`.

Para efectos de esta tarea, un polinomio es válido si cada uno de los términos que lo compone tiene un exponente diferente. Además, un polinomio puede tener coeficientes nulos (iguales a 0) y estar en cualquier orden. Por ejemplo:

- $2x + 4x^5 + 5$ es un polinomio válido.
- $3x^3 + 0x + 2x^2$ es un polinomio válido.
- $7x^4 + 2x^4 + 3x^{10}$ no es un polinomio válido (tiene 2 términos con exponente 4).

Para las siguientes preguntas asuma que los polinomios serán válidos.

(a) [8 Pt] A partir de las reglas anteriores defina el tipo de datos recursivo `Poly` y acompañelo de su gramática.

(b) [4 Pt] Usando recursión explícita, defina la función

```
;; degree :: Poly -> Integer
```

que indica el grado de un polinomio (el exponente mayor). Recuerde que un término con coeficiente nulo no influye en el grado de un polinomio. Además, para el caso del polinomio nulo (`nullp`), la función debe arrojar el mensaje de error: “El polinomio nulo no tiene grado”.

- (c) [4 Pt] Usando recursión explícita, defina la función

```
;; coefficient :: Integer Poly -> Integer
```

que entrega el coeficiente numérico asociado a un exponente dado. Por ejemplo:

```
>>> (coefficient 2 (plus 10 2 (plus 3 1 (nullp))))
10
>>> (coefficient 3 (plus 10 2 (plus 3 1 (nullp))))
0
```

- (d) [6 Pt] Se dice que un polinomio está en *forma normal* si sus exponentes están ordenados de mayor a menor y se omiten aquellos exponentes con coeficiente nulo. A partir de ello, defina usando recursión explícita la función

```
;; nf? :: Poly -> Bool
```

que indica si un polinomio se encuentra en forma normal.

- (e) [6 Pt] Usando recursión explícita, defina la función

```
;; normalize :: Poly -> Poly
```

que normaliza un polinomio. *Hint:* Puede ser de utilidad separar el proceso de normalización en funciones auxiliares.

- (f) [6 Pt] Defina la función

```
;; eval :: Integer Poly -> Integer
```

que evalúa un polinomio con un punto dado. Por ejemplo:

```
>>> (eval 2 (plus 10 2 (plus 3 1 (nullp))))
46
```

- (g) [6 Pt] Usando recursión explícita, defina la función

```
;; map-poly :: (Integer Integer -> Integer * Integer) Poly -> Poly
```

que aplica una función sobre cada uno de los términos del polinomio. Por ejemplo, a continuación se muestra la aplicación de una función que duplica el valor de los coeficientes numéricos:

```
>>> (map-poly (λ(c d) (cons (* c 2) d))
      (plus 10 2 (plus 3 1 (nullp))))
(plus 20 2 (plus 6 1 (nullp)))
```

- (h) [8 Pt] Defina la función

```
;; fold-poly :: A (Integer Integer A -> A) -> (Poly -> A)
```

que captura el esquema de recursión asociado a un polinomio.

- (i) [12 Pt] Utilizando fold-poly defina las funciones: `coefficient2`, `map-poly2` y `eval2` (4 Pt. c/u). Estas funciones deben tener el mismo comportamiento que aquellas definidas con recursión explícita.